Demystifying

Spring Boot Magic

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com



Abstract

Demystifying Spring Boot Magic

Spring Boot often feels like magic. You add a dependency, run your application, and suddenly everything works. Until something breaks and the magic turns into a black box.

We take a deep dive into auto-configuration, the core of Spring Boot's convention-over-configuration model. You will see how Spring Boot decides what to configure by exploring mechanisms such as @Conditional, auto-configuration ordering, and the metadata that drives these decisions.

By the end, you will be able to trace why auto-configuration applies or does not, debug unexpected behavior, and confidently adapt Spring Boot to your needs.

Demystifying Spring Boot Magic



Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Introduction



Patrick Baumgartner

Technical Agile Coach @ **42talents**

My focus is on the **development of software solutions *with* humans**.

Working with Spring since 2008 and teaching courses since 2011.

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of Voxxed Days Zurich, **JUG Switzerland**, **Java Champion**, Oracle ACE Pro Java ...

@patbaumgartner

A Quick Show of Hands

- 🙋 Who has ever added a dependency and been **surprised** by what started working automatically?
- 🙋 Who has debugged a Spring Boot issue without fully understanding what changed?
- 🙋 Who has wondered: *"Where does this bean actually come from?"*

> Spring Boot is **opinionated**. This talk is about understanding those opinions.


I
DO
NOT
CARE

Agenda

Agenda

- Magic vs Boilerplate
- Why Spring Boot Keeps Winning
- Dependency Management & Modular Starters
- AutoConfiguration
- Externalized Configuration
- `@SpringBootTest` and FailureAnalyzer
- Fat JARs & Buildpacks
- Spring Boot DevTools
- Actuator: X-Raying the Magic
- Custom Starters



Magic vs Boilerplate

What Developers Mean by "Magic"

- Code behavior at runtime is unclear from reading the source alone.
- Violations of *Single Responsibility* or *Command/Query Separation* often feel like "magic".
- The line between "magic" and understandable code is thin. It’s often about how surprising the behavior is.

What Spring Boot Eliminated

Before Spring Boot, every service needed this:

```
@Bean DataSource dataSource() {
    HikariDataSource ds = new HikariDataSource();
    ds.setJdbcUrl(env.getProperty("spring.datasource.url"));
    ds.setUsername(env.getProperty("spring.datasource.username"));
    ... // more properties
    return ds;
}

@Bean EntityManagerFactory entityManagerFactory(DataSource ds) { ... }

@Bean PlatformTransactionManager transactionManager() { ... }
```

For each service. Multiplied by dozens. **That is boilerplate.**

Spring Boot hides it behind convention. But *how*?

Magic or Boilerplate?

`@SpringBootApplication` - where does your `DataSource` come from?

Neither. It's a Convention.

- Spring Boot removes boilerplate through **auto-configuration**.
- Auto-configuration follows explicit, documented, overridable rules.
- When you don't know the rules, it feels like a black box.
- When you do, it feels like a superpower.

> *Understanding the rules turns magic into knowledge.*

Too Much Magic

"Boilerplate is mostly a problem when writing code. Magic when reading & debugging. We read & debug more than we write."

David Tanzer - @dtanzer



When Magic Becomes Knowledge

"It is too much magic until you read the docs, several books, and the source code, after that, the magic becomes knowledge."

Unknown





The Spring Boot Contract

- **Classpath-driven** - what is on the classpath shapes what runs.
- **Opinionated defaults** - every default is overridable with one starter, dependency, property, or `@Bean` .
- **Backoff by design** - as soon as you define your own bean, Boot steps aside gracefully.
- **Production-ready built in** - health, metrics, externalized config come with every app.

> If you follow the conventions, you get functionality for free. Break the contract **knowingly**, not by accident.

Spring Boot Keeps Winning

The Numbers

- **~1 billion downloads per year** across the Spring ecosystem.
- Used in **60%+ of Java back-end services** in enterprise surveys.
- More than **170 official Spring Boot starters** - integrations for nearly everything.

> Spring isn't winning because the alternatives are bad. It's winning because **the ecosystem keeps compounding**.

Spring Boot Keeps Winning

Convention, Cognitive Load, and Boring Technology

"A good framework is invisible. It only shows up when something breaks."

Unknown

"Every novel technology choice drains your innovation tokens. Choose the boring solution first."

Dan McKinley, Blog - Choose Boring Technology



Spring Boot Keeps Winning

Evolution Without Revolution

- Spring Boot **1.x through 4.x** - same mental model, modern internals.
- Migration tooling ships with every major version:
  - **`spring-boot-properties-migrator`** detects renamed properties automatically.
  - **OpenRewrite `rewrite-spring`** recipes automate Java and config migrations.
- The Spring team publishes **release notes and migration guides** before GA.
- Long-term support windows backed by VMware / Broadcom.

> "Stability is a feature." - *Michael Nygard, Release It!*


SNAKE
OIL

Spring Boot Dependency Management

Spring Boot Parent POM

- Defines versions of key dependencies.
- Uses `dependencyManagement` internally via the `spring-boot-dependencies` parent.
- Preconfigures common Maven plugins.
- Sets the Java version for the project.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>
```

Spring Boot “Starter” Dependencies

- Provide an easy way to include multiple coordinated dependencies.
- Automatically pull in required **transitive dependencies**.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
    <!-- NO VERSION NEEDED -->
  </dependency>
</dependencies>
```

- Versions defined in `spring-boot-dependencies`

```
<tomcat.version>11.0.22</tomcat.version>
```

See also: https://repo1.maven.org/maven2/org/springframework/boot/spring-boot-dependencies/4.1.0/spring-boot-dependencies-4.1.0.pom

Dependency Management with Gradle

- The Dependency Management Plugin ensures consistent, predefined versions for project dependencies.

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.0'
    id 'io.spring.dependency-management' version '1.1.7'
}
```

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

```
ext['tomcat.version'] = '11.0.22'
```

Modularized Spring Boot

One Big Jar → 50+ Focused Modules

- Spring Boot ships **small, focused modules** instead of several large jars.
- Naming is now predictable:
  - Module: `spring-boot-<technology>` (e.g. `spring-boot-jdbc`)
  - Package: `org.springframework.boot.<technology>`
  - Starter: `spring-boot-starter-<technology>`
  - Test starter: `spring-boot-starter-<technology>-test`
- Some things now **need a starter** that didn't before (e.g. Flyway, Liquibase).

See: The Spring Team - "Modularizing Spring Boot", https://spring.io/blog/2025/10/28/modularizing-spring-boot/



@SpringBootApplication

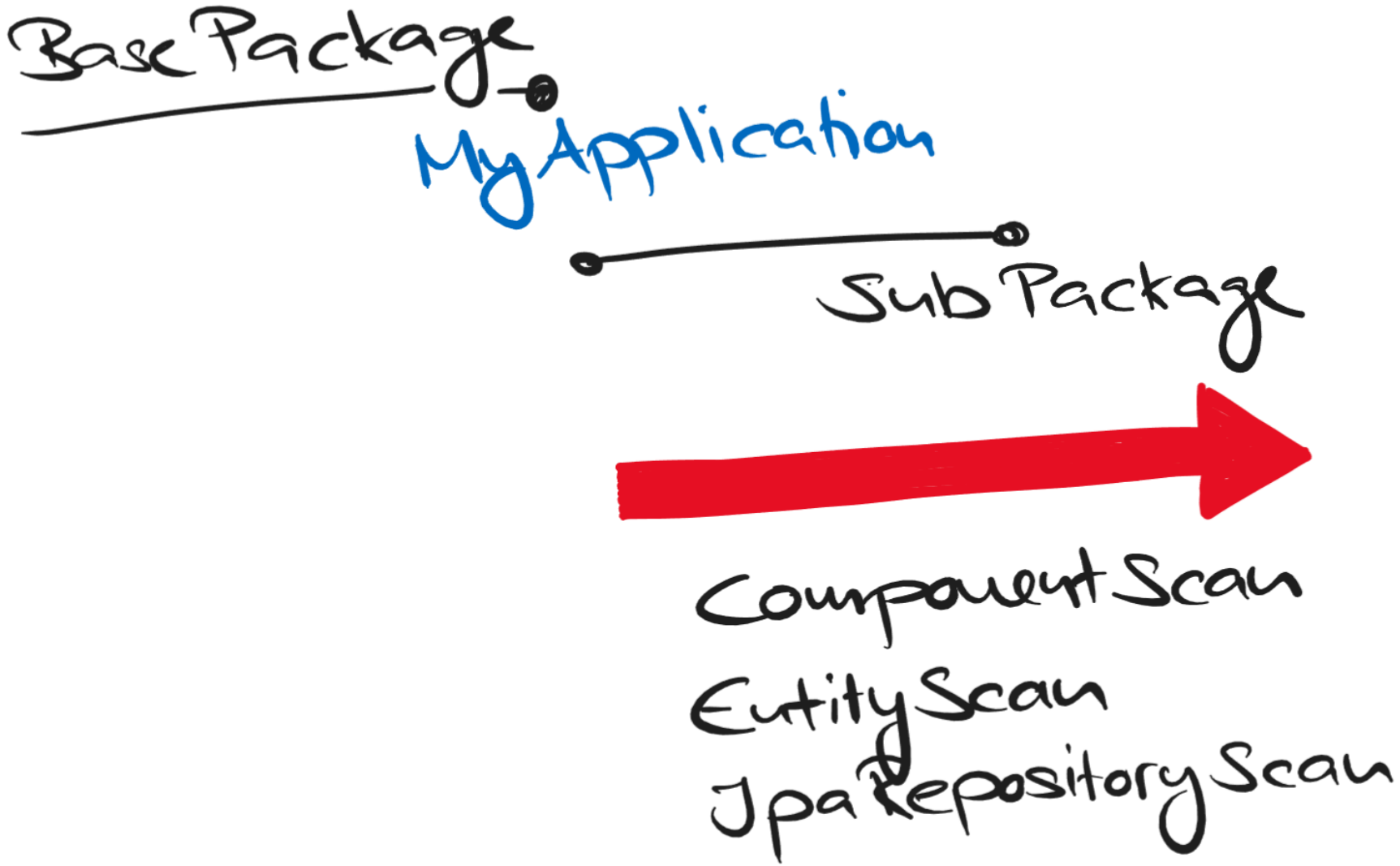
Auto Configuration

- Follows the convention over configuration principle.
- Can feel like "magic" due to hidden behaviors.

```
@SpringBootApplication
public class MyApplication { }
```

- Is shorthand for:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class MyApplication { }
```


Base Package
MyApplication
Sub Package
ComponentScan
EntityScan
JpaRepositoryScan

@AutoConfiguration or "@Abracadabra"

@AutoConfiguration

A specialised `@Configuration` with built-in opinions:

- Sets **`proxyBeanMethods = false`** - no CGLIB subclassing, faster startup
- Carries ordering metadata: `before` / `after` other auto-configurations
- Is **package-private** by design - never part of any public API
- Only activated when its `.imports` file is loaded from the classpath

```
@AutoConfiguration(after = DataSourceAutoConfiguration.class)
class JdbcTemplateAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }
}
```

@AutoConfiguration

A specialised `@Configuration` with built-in opinions:

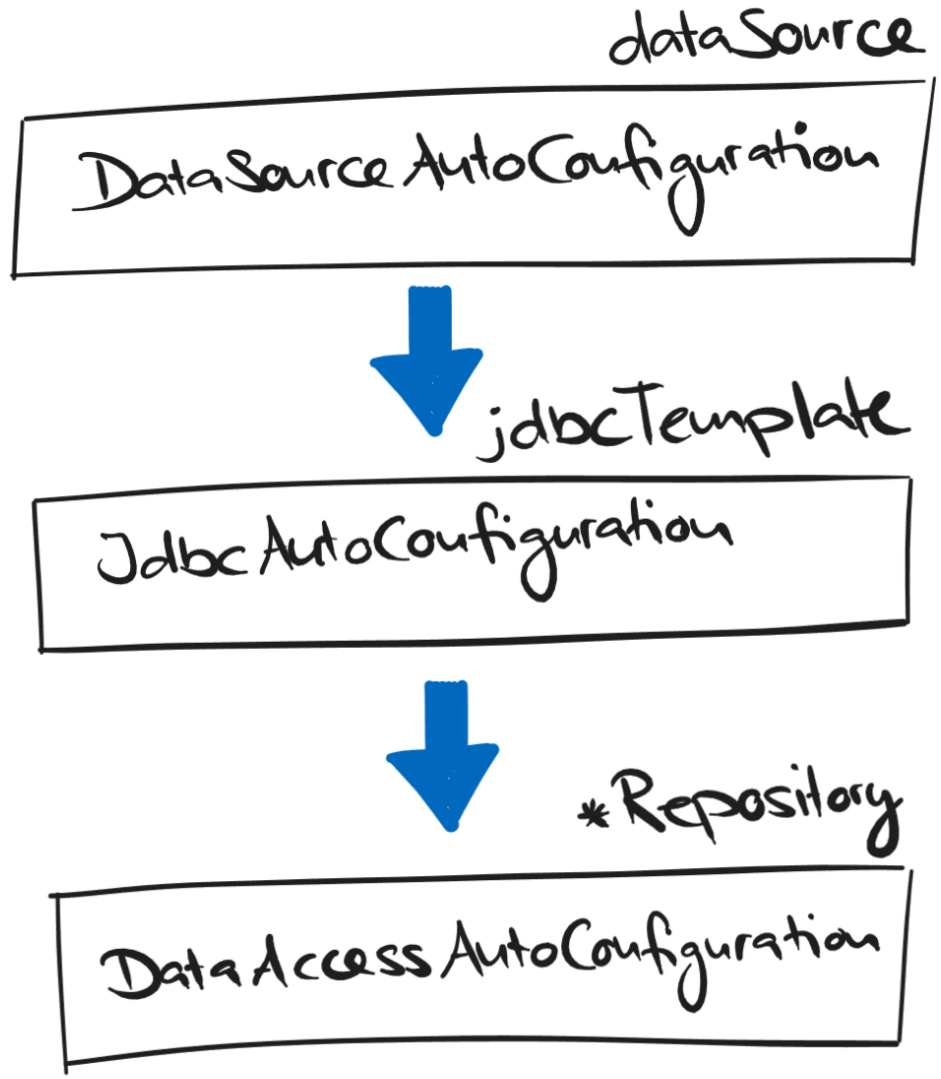
- Sets **`proxyBeanMethods = false`** -r
- Carries ordering metadata: l
- Is **package-private** l
- Only activa

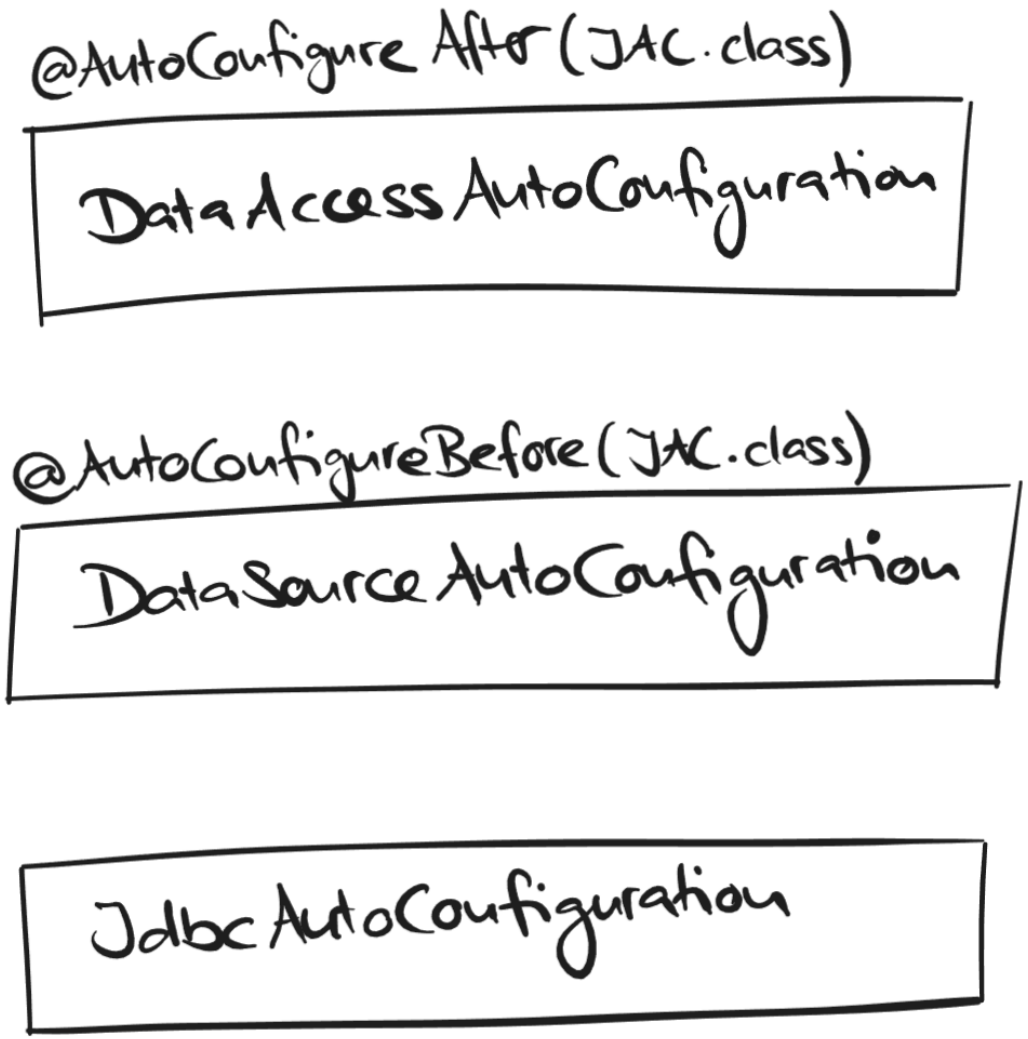
Tip: Set `proxyBeanMethods = false` when beans don't call each other. Removes CGLIB proxy, speeds up startup.

```
@
                    eAutoConfiguration.class)
                tion {

        @
            nMissingBean
    tate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }
}
```

@AutoConfigureBefore / @AutoConfigureAfter


dataSource
DataSourceAutoConfiguration
jdbcTemplate
JdbcAutoConfiguration
*Repository
DataAccessAutoConfiguration


@AutoConfigureAfter(JAC.class)
DataAccessAutoConfiguration
@AutoConfigureBefore(JAC.class)
DataSourceAutoConfiguration
JdbcAutoConfiguration

Import Candidates in META-INF/spring/

org.springframework.boot.autoconfigure.AutoConfiguration.imports

- **Every module ships its own** `.imports` file - no single big list.

```
# spring-boot-jdbc.jar -> META-INF/spring/…AutoConfiguration.imports
org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
org.springframework.boot.jdbc.autoconfigure.JdbcClientAutoConfiguration
org.springframework.boot.jdbc.autoconfigure.JdbcTemplateAutoConfiguration
org.springframework.boot.jdbc.autoconfigure.DataSourceTransactionManagerAutoConfiguration
```

```
# spring-boot-flyway.jar -> META-INF/spring/…AutoConfiguration.imports
org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
```

```
# spring-boot-hibernate.jar -> META-INF/spring/…AutoConfiguration.imports
org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
```

- No starter on the classpath → no imports file → nothing to evaluate. **Faster and more explicit.**

Auto Configuration

```
package org.springframework.boot.jdbc.autoconfigure;

@AutoConfiguration(before = SqlInitializationAutoConfiguration.class)
@ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class })
@ConditionalOnMissingBean(type = "io.r2dbc.spi.ConnectionFactory")
@EnableConfigurationProperties(DataSourceProperties.class)
@Import({ DataSourcePoolMetadataProvidersConfiguration.class, DataSourceCheckpointRestoreConfiguration.class })
public final class DataSourceAutoConfiguration {

        @Configuration(proxyBeanMethods = false)
        @Conditional(EmbeddedDatabaseCondition.class)
        @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
        @Import(EmbeddedDataSourceConfiguration.class)
        static class EmbeddedDatabaseConfiguration {
        }

        @Configuration(proxyBeanMethods = false)
        @Conditional(PooledDataSourceCondition.class)
        @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
        @Import({ DataSourceConfiguration.Hikari.class, DataSourceConfiguration.Tomcat.class,
                        DataSourceConfiguration.Dbcp2.class, DataSourceConfiguration.OracleUcp.class,
                        DataSourceConfiguration.Generic.class, DataSourceJmxConfiguration.class })
        static class PooledDataSourceConfiguration {
```

Conditions

`@Conditional` is the engine behind all of Spring Boot's decisions:

- `@ConditionalOnClass / OnMissingClass`
- `@ConditionalOnBean / OnMissingBean`
- `@ConditionalOnProperty / OnBooleanProperty`
- `@ConditionalOnResource`
- `@ConditionalOnExpression`
- `@ConditionalOnJava`
- `@ConditionalOnWebApplication / OnNotWebApplication`
- `@ConditionalOnCloudPlatform`

Conditions

`@Conditional` is the engine behind all of Sp

- `@ConditionalOnClass / On`
- `@ConditionalOnBea`
- `@Conditiona`
- `@Con`

`ation / OnNotWebApplication`

`oudPlatform`

Lesser known: `@ConditionalOnSingleCandidate` activates only when exactly one primary bean of that type exists. Two `DataSource` beans? Boot backs off silently.

Magic or Knowledge? 🎲

You add `spring-boot-starter-data-jpa` and the H2 driver.
No config at all. Does the app start?

Knowledge!

- **Yes.** `DataSourceAutoConfiguration` finds H2 on the classpath → `EmbeddedDatabaseCondition` matches → in-memory `DataSource`.
- `HibernateJpaAutoConfiguration` runs **after** it (`@AutoConfigureAfter`) and finds the `DataSource` bean.
- Define your own `DataSource` bean and both back off: `@ConditionalOnMissingBean`.

The magic is just **conditions evaluated in a defined order.**

Did You Know? Everything is `@Conditional`

`@Profile` is not special. Look inside:

```
@Conditional(ProfileCondition.class)
public @interface Profile { String[] value(); }
```

Every "smart" Spring annotation is just a `@Conditional` in disguise:

Annotation	Underlying condition
`@Profile("prod")`	`ProfileCondition`
`@ConditionalOnClass`	`OnClassCondition`
`@ConditionalOnBean`	`OnBeanCondition`
`@ConditionalOnProperty`	`OnPropertyCondition`

Did You Know? Everything is @Conditi

`@Profile` is not special. Look inside:

```
@Conditional(ProfileCondition.class)
public @interface Profile { Stri
```

Every "smart" Sprin

Annota	
	sCondition
	OnBeanCondition
roperty	OnPropertyCondition

Tip: Write your own `Condition`. Implement `Condition`, return true/false, annotate with `@Conditional(YourCondition.class)`. That is the entire API.



Externalized Configuration

Externalized Configuration (Simplified)

Configuration properties loaded in a specific order

1. Devtools settings
2. `@TestPropertySource` and `@SpringBootTest` properties
3. Command line arguments
4. `SPRING_APPLICATION_JSON` (inline JSON properties)
5. ServletConfig / ServletContext parameters
6. JNDI attributes from `java:comp/env`
7. Java System properties (`System.getProperties()`)
8. OS environment variables
9. Profile-specific application properties
10. Application properties / YAML
11. `@PropertySource` files
12. `SpringApplication.setDefaultProperties`

Externalized Configuration (Simplified)

Configuration properties loaded in a specific o

1. Devtools settings
2. `@TestPropertySource` and `@Spr`
3. Command line arguments
4. `SPRING_APPLICATIO`
5. ServletConfig
6. JNDI at
7. `stem.getProperties()`)
   - ables
   - application properties
11. `@PropertySource` files
12. `SpringApplication.setDefaultProperties`

Tip: `SPRING_APPLICATION_JSON` works in containers. An entire config file in a single env var.

@ConfigurationProperties

@ConfigurationProperties

- Binds external properties to a typed bean using a *prefix*. Works with classes and **records**.

```java
// Classic style
@ConfigurationProperties(prefix = "tenant.connection")
public class ConnectionProperties {
    private String hostUrl;
    private int port = 5432;
    public String getHostUrl() { return hostUrl; }
    public void setHostUrl(String h) { this.hostUrl = h; }
}
```

```java
// Record style (Spring Boot 2.6+) - immutable, no boilerplate
@ConfigurationProperties(prefix = "tenant.connection")
public record ConnectionProperties(String hostUrl, int port) {}
```

```
tenant.connection.host-url=db.internal
tenant.connection.port=5432
```

Magic or Knowledge? 🎲

Which of these keys binds to `hostUrl`?

```
@ConfigurationProperties("tenant.client-connection")
public class ConnectionSettings {
    private String hostUrl;
    …
}
```

- A: `tenant.clientConnection.hostUrl`
- B: `tenant.client-connection.host-url`
- C: `tenant.client_connection.host_url`
- D: `TENANT_CLIENTCONNECTION_HOSTURL`

Knowledge! All of them.

Relaxed Binding in @ConfigurationProperties

- Supports flexible mapping between property keys and Java fields.
- Valid property keys for `hostUrl`:
  - `tenant.clientConnection.hostUrl`
  - `tenant.client-connection.host-url`
  - `tenant.client_connection.host_url`
  - `TENANT_CLIENTCONNECTION_HOSTURL`
- Canonical form in code and docs: **kebab-case** (`host-url`).

@SpringBootTest

@SpringBootTest

- Enables integration tests with the full Spring Boot context.
- Starts the real application, detects `@SpringBootConfiguration` automatically.
- Supports `webEnvironment` modes: `MOCK`, `RANDOM_PORT`, `DEFINED_PORT`, `NONE`.

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class ClaimControllerIT {
    @Autowired TestRestTemplate rest;
    // ...
}
```

@SpringBootTest

- Enables integration tests with the full Spring Bo
- Starts the real application, detects @Spr
  automatically.
- Supports webEnvironm
  DEFINED_PORT

_PORT ,

```
@Spring
Cl                                    otTest.WebEnvironment.RANDOM_PORT)
                                      Test;
```

Tip: Use @SpringBootTest judiciously. Overuse is the #1 cause of slow test suites.

Test Slices - Boot's Best Kept Secret

Test Slices

Full context is the sledgehammer. Use the scalpel:

`@WebMvcTest`	Web layer only - controllers, filters, security
`@DataJpaTest`	JPA repositories + in-memory DB
`@RestClientTest`	HTTP clients + `MockRestServiceServer`
`@JsonTest`	JSON serialization / deserialization
`@DataRedisTest`	Redis repositories only

```
@WebMvcTest(ClaimController.class)
class ClaimControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean ClaimService service; // only the web layer is loaded
}
```

FailureAnalyzer

Exceptions All the Way...

```
at org.springframework.beans.factory.support.DefaultListableBeanFactory.doResolveDepe...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveDepend...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor$...
... 226 common frames omitted
Caused by: org.springframework.beans.factory.BeanCreationException: Could not autowire fi...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor$...
at org.springframework.beans.factory.annotation.InjectionMetadata.inject(InjectionMet...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor....
... 246 common frames omitted
Caused by: org.springframework.beans.factory.BeanCurrentlyInCreationException: Error crea...
at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.beforeSingl...
at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleto...
at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBe...
at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBean...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.findAutowireC...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.doResolveDepe...
at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveDepend...
at org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor$...
... 248 common frames omitted
```

FailureAnalyzer

- Handles specific exceptions at startup and prints a **human-readable diagnosis** with an action.
- Built-in: `NoSuchBeanDefinition`, `PortInUse`, `DataSourceBeanCreation`, `BeanCurrentlyInCreation`, `Bind`, …

```
// Build your own in 5 lines:
public class MissingConfigAnalyzer
        extends AbstractFailureAnalyzer<MissingConfigException> {

    @Override
    protected FailureAnalysis analyze(Throwable cause, MissingConfigException ex) {
        return new FailureAnalysis(
            "Required configuration key '" + ex.getKey() + "' is not set.",
            "Add '" + ex.getKey() + "' to your application.properties.",
            ex);
    }
}
```

Fat JARs

Fat JARs

- The concept isn't new:
  - Also known as Uber-JAR or One-JAR.
  - Popularized by tools like the Apache Maven Shade Plugin.
- Spring Boot simplifies it via the Spring Boot Maven Plugin:

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>4.1.0</version>
    <executions>
        <execution>
            <goals>
                <goal>repackage</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

Exploded Fat JARs

- Fat JARs can be an anti-pattern in containerized environments.
- They lead to large, non-cacheable image layers instead of thin, cacheable ones.
- Thin layers reduce build time and network traffic, and improve Docker cache efficiency.

```
$ du -sh target/demo-0.0.1-SNAPSHOT*
14M  target/demo-0.0.1-SNAPSHOT.jar
4.0K target/demo-0.0.1-SNAPSHOT.jar.original
```

- Layertools allow you to extract the layers from a fat JAR.

```
java -Djarmode=tools -jar demo-0.0.1-SNAPSHOT.jar extract --layers --launcher
```



paketo

buildpacks

Buildpacks FTW!

- Spring Boot Maven Plugin uses Cloud Native Buildpacks via the `build-image` goal.

```
# One command. No Dockerfile.
./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=claims-service:latest
```

- **Runtime optimizations:** exploded layers, memory tuning at container start.
- **Spring AOT:** pre-generates reflection metadata - 30-50% faster startup.
- **JVM AOT Cache (Java 25+):** buildpack performs a training run and bakes an `.aot` cache into the image - JVM class loading skipped on every restart.
- **Incremental builds:** only changed layers are rebuilt.

```
# Enable JVM AOT Cache in the image (requires Java 25+ / Paketo buildpack):
./mvnw spring-boot:build-image \
  -Dspring-boot.build-image.imageName=claims-service:latest \
  -Dspring-boot.build-image.env.BP_JVM_AOTCACHE_ENABLED=true
```

Spring Boot DevTools

Spring Boot DevTools

- Applies development-friendly defaults (e.g. disables template caching).
- Starts an embedded LiveReload server.
- Supports a global config in `$HOME/.config/spring-boot` .
- Automatically restarts the app when classpath files change.
- Supports remote debugging with `RemoteSpringApplication` .

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <optional>true</optional>
  <scope>runtime</scope>
</dependency>
```

Spring Boot DevTools

- Applies development-friendly defaults (e.g. disabl
- Starts an embedded LiveReload server.
- Supports a global config in `$HOME/`
- Automatically restarts the a ge.
- Supports remote de gApplication .

```
<dependency>
  <groupI
  <a                    d>
                        tifactId>
```

Lesser known: DevTools uses two classloaders. Yours restarts, libraries stay warm. 2-5x faster than a cold start.

Actuator: X-Raying the Magic

/actuator/conditions

- The **condition evaluation report** - at runtime, no restart needed.
- Shows every auto-configuration: matched, not matched, and **why**.

```
"DataSourceAutoConfiguration": [
  {
    "condition": "OnClassCondition",
    "message": "@ConditionalOnClass found required classes
                'javax.sql.DataSource', 'org.springframework.jdbc…'"
  }
]
```

- Same report at startup: run with `--debug` .

More X-Ray Endpoints

- `/actuator/beans` - every bean, its type, scope, and dependencies.
- `/actuator/configprops` - all `@ConfigurationProperties` with resolved values.
- `/actuator/env` - every property and **which property source it came from**.
- Values are sanitized by default - `password`, `secret`, `key` masked as `******`.

```
// Customise what gets masked:
@Bean
SanitizingFunction customSanitizer() {
    return data -> data.getKey().contains("ssn")
        ? data.withValue("[PROTECTED]")
        : data;
}
```

Production Features by Default

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

- **Liveness and readiness probes enabled by default** (Spring Boot 4.x):
  - `/actuator/health/liveness` - is the app alive?
  - `/actuator/health/readiness` - is the app ready to serve traffic?
- Kubernetes-ready without any configuration.
- Opt out: `management.endpoint.health.probes.enabled=false`
- Expose only what you need:

```properties
management.endpoints.web.exposure.include=health,info,conditions,env,configprops
management.endpoint.health.show-details=always
```

Magic or Knowledge? 🎲

This is a plain Java interface. No implementation anywhere. Can you inject it with `@Autowired`?

```
@HttpExchange(url = "https://claims-api.internal")
public interface ClaimsClient {

    @GetExchange("/claims/{id}")
    Claim findById(@PathVariable String id);

    @PostExchange("/claims")
    Claim submit(@RequestBody ClaimRequest request);
}
```

Knowledge! Lesser-Known Goodies

- **HTTP Service Clients**: Boot generates an implementation for `@HttpExchange` interfaces at startup. Configure via `spring.http.serviceclient.*`.
- **Lazy JDBC connections**: `spring.datasource.connection-fetch=lazy` - pool slot taken only when a statement actually executes.
- **`@Fallback` beans** (Spring Framework 6.2+): Mark a bean as fallback - it is registered in the context but only injected when no other primary candidate exists. Different from `@ConditionalOnMissingBean`; the fallback is **always** in the context.

SSRF Hardening - Built In

`InetAddressFilter` (Spring Boot 4.1)

- Blocks outgoing HTTP requests to **internal / private addresses** by default.
- Critical for services that accept user-supplied URLs (webhooks, integrations).

```
// Automatic - nothing to wire. Override only to allow exceptions:
@Bean
InetAddressFilter inetAddressFilter() {
    InetAddressFilter filter = new InetAddressFilter();
    filter.setAllowedAddresses(List.of("10.0.0.0/8")); // internal mesh only
    return filter;
}
```

```
# Opt out entirely (not recommended in production):
spring.webmvc.inet-address-filter.enabled=false
```

OpenTelemetry - One Dependency

`spring-boot-starter-opentelemetry` (Spring Boot 4.1)

- Exports **metrics + traces** over OTLP with zero extra wiring.
- Replaces the need for manual Micrometer + Zipkin / Jaeger configuration.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-opentelemetry</artifactId>
</dependency>
```

```
spring.application.name=claims-service
management.otlp.tracing.endpoint=http://otel-collector:4318/v1/traces
management.otlp.metrics.export.url=http://otel-collector:4318/v1/metrics
```



Recap

Recap

AutoConfiguration is not magic!

- `@SpringBootApplication` enables auto-configuration.
- Uses ImportCandidates instead of classpath scanning.
  - one `.imports` file **per module**.
- `@ConditionalOnClass` and `@ConditionalOnBean` enable flexible, overridable behavior.
- Auto-configurations run after user-defined beans.
- Control ordering with `@AutoConfigureBefore` and `@AutoConfigureAfter` .

Close the Loop 🙋

At the start of this talk you raised your hand.

- Something about Spring Boot surprised you.
- Something broke and you didn't know why.
- You wondered where a bean came from.

Close the Loop 🙋

Hands up again - can you now explain it?

- Is it a `@ConditionalOnClass` that matched because a JAR landed on the classpath?
- A `@ConditionalOnMissingBean` that stepped aside once you added your own?
- A `.imports` file that activated a whole tree of configuration?
- A property from the wrong source winning the priority battle?

Close the Loop 🙋

The answer is always in the condition evaluation report.

- `--debug` at startup.
- `/actuator/conditions` at runtime.

> The magic was always documented. Now you know where to read.

Your Debugging Toolbox

Startup, why was a bean (not) created?	`--debug` → condition evaluation report
At runtime, no restart	`/actuator/conditions`
Where does this property value come from?	`/actuator/env` , `/actuator/configprops`
Startup fails with a wall of stack traces	Read the `FailureAnalyzer` output first
Upgrading and properties moved?	`spring-boot-properties-migrator` or **OpenRewrite** `rewrite-spring`



Creating a Custom Starter

Why Create a Custom Starter?

Imagine you ship **10 new services** over the next year. Each one needs compliant **audit logging** - who did what, when, on which entity. Without a starter, that's copy-paste boilerplate in every repo.

A custom starter helps you:

- **Standardize compliance behavior** across all services
- **Package reusable functionality** into one dependency
- **Simplify dependency and version management**
- **Enable plug-and-play features** via auto-configuration
- **Improve developer experience** with minimal setup

Anatomy of a Custom Starter

```
audit-spring-boot/
├── audit-spring-boot-autoconfigure/
│   ├── src/main/java/com/fortytwotalents/audit/
│   │   ├── AuditEvent.java
│   │   ├── AuditEventListener.java
│   │   └── autoconfigure/
│   │       ├── AuditAutoConfiguration.java
│   │       └── AuditProperties.java
│   ├── src/main/resources/META-INF/spring/
│   │   └── ...AutoConfiguration.imports
│   └── pom.xml
└── audit-spring-boot-starter/
    └── pom.xml   <- just a POM, no Java code
```

Add `spring-boot-configuration-processor` to `autoconfigure` for IDE property auto-completion.

Domain: AuditEvent + AuditEventListener

```java
package com.fortytwotalents.audit;

public record AuditEvent(String principal, String action, String resourceId) {}
```

```java
package com.fortytwotalents.audit;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.event.EventListener;

public class AuditEventListener {

    private static final Logger log = LoggerFactory.getLogger(AuditEventListener.class);
    private final String topic;

    public AuditEventListener(String topic) { this.topic = topic; }

    @EventListener
    public void on(AuditEvent event) {
        log.info("[AUDIT] topic={} principal={} action={} resource={}",
```

AuditProperties + AuditAutoConfiguration

```java
package com.fortytwotalents.audit.autoconfigure;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "audit")
public record AuditProperties(boolean enabled, String topic) {
    public AuditProperties {
        if (topic == null || topic.isBlank()) { topic = "audit-events"; }
    }
}
```

```java
package com.fortytwotalents.audit.autoconfigure;

import com.fortytwotalents.audit.AuditEventListener;
import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.context.annotation.Bean;
```

Registration + Starter POM

```
# audit-spring-boot-autoconfigure/src/main/resources/
# META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.fortytwotalents.audit.autoconfigure.AuditAutoConfiguration
```

```
<!-- audit-spring-boot-starter/pom.xml - NO source code here! -->
<dependencies>
  <dependency>
    <groupId>com.fortytwotalents</groupId>
    <artifactId>audit-spring-boot-autoconfigure</artifactId>
    <version>${project.version}</version>
  </dependency>
  <!-- add your library dependencies here if needed -->
</dependencies>
```

The starter is **just a POM**. The autoconfigure module does the work.

Using the Starter

Consumers only need one dependency - auto-configuration does the rest.

```xml
<dependency>
  <groupId>com.fortytwotalents</groupId>
  <artifactId>audit-spring-boot-starter</artifactId>
  <version>1.0.0</version>
</dependency>
```

```java
// Publish an audit event from any service layer:
publisher.publishEvent(new AuditEvent("user:42", "CLAIM_APPROVED", claimId));
```

```properties
# application.properties - nothing required, works out of the box
# Opt out on a specific service:
audit.enabled=false

# Route to a specific topic:
audit.topic=claims-audit-events
```

Community

Community

- **Spring.io** - docs, guides, official blog - spring.io/blog
- **GitHub** - github.com/spring-projects
  - Issues and milestones: the best place to understand Boot internals
- **YouTube** - Josh Long and Dan Vega
- **Bluesky / Mastodon** - follow the Spring team
- **Meetups** - Spring I/O (Barcelona), Devoxx, Voxxed Days Zurich, JUG Switzerland

> The community is where magic becomes knowledge. Show up.

Q&A

Questions? Thoughts? Unpopular opinions?

Patrick Baumgartner
@patbaumgartner
patrick.baumgartner@42talents.com



Demystifying Spring Boot Magic



Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com